STOCK MATRIX ANALYZER · PROJECT OVERVIEW

How Ask AI answers a question

Ask AI is the chat tab of a portfolio app. You ask in plain English; it answers only from your holdings, SEC filings, analyst data and live prices, cites every figure, and says “I don't know” rather than guess. This page explains its four paths, its agents and tools, and the guardrails that keep answers honest.

4
paths a question can take

6
LLM agents

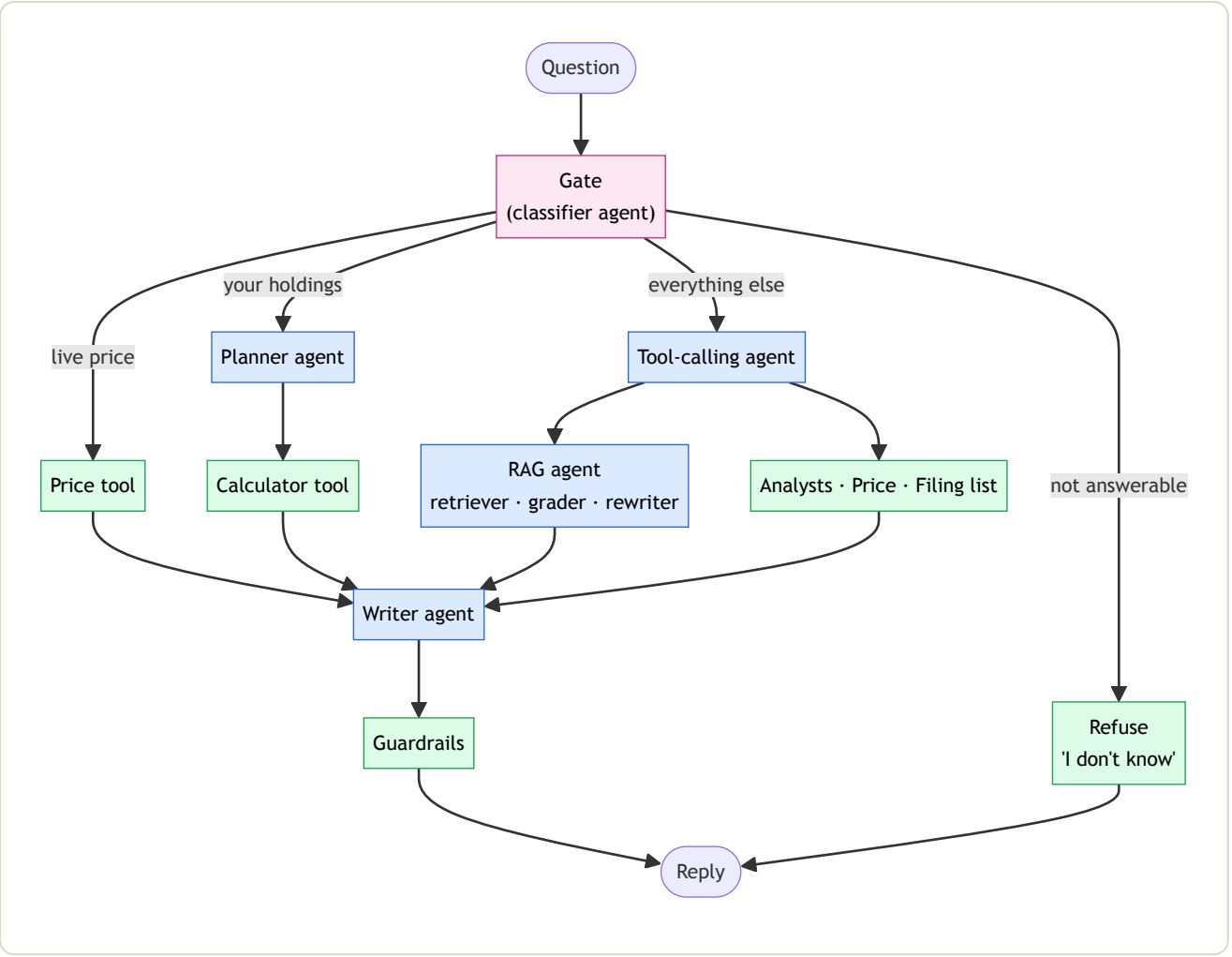
5
agent tools

4,395
filing chunks per index (AAPL, RIVN)

364
automated tests

The four paths

Every question first reaches the **gate**, a classifier agent. It decides which of four paths the question needs. Two paths are quick and direct; one is a flexible agent; one politely refuses.



`pink` classifier agent `blue` LLM agents `green` tools and checks (plain code)

Path	Used for	What runs	Typical time
Refuse	Advice (“should I buy?”), price predictions, news, unknown companies	Gate, then a polite “I don't know” that names the part it won't answer	about 2 s
Live price	“What's Apple's price?”	Gate, price tool, writer	about 4 s
Portfolio	Questions about your own holdings: values, losses, weights, what-ifs	Gate, planner picks up to 3 calculations, calculator, writer	about 6 s
Agent	Everything else: filings, analysis, outlook, mixed questions	Gate, tool-calling agent with its tools (including the RAG agent), writer	10–17 s

The agents

An agent here is an LLM given one job, a fixed instruction (system prompt) and, for most, a fixed answer format. The LLMs only decide and write; every number is computed in code.

Agent	Type	Its one job	Model
Gate	`classifier`	Picks the path, finds tickers (even misspelled: “rivianll” → RIVN), flags outlook and filing questions, notes any part it won't answer	Jev router via OpenRouter
Planner	`planner`	Turns a holdings question into up to 3 calculator calls	OpenAI gpt-4o-mini
Tool-calling agent	`orchestrator`	Chooses tools for agent-path questions (max 6 calls, 4 turns)	OpenAI gpt-4o-mini
Grader	`RAG`	Keeps only filing passages that truly answer the question	OpenAI gpt-4o-mini
Rewriter	`RAG`	Rewords the search once if nothing relevant was found	OpenAI gpt-4o-mini
Writer	`generation`	Writes the reply from the evidence, with a citation on every figure	OpenAI gpt-4o-mini

The tools

Tools are plain code. They fetch or compute facts and return them with a label the writer must cite, such as `[AAPL 10-K, 2025-10-31, Item 1A]`.

Tool	What it does	Data source
search_filings	The RAG search over SEC filings (details below)	Pinecone
list_filings	Lists which filings exist and the latest of each form, with SEC links	Pinecone metadata
portfolio_calc	Exact holding value, gain/loss, weights, top-N lists, totals and what-if shocks	Your uploaded holdings

Tool	What it does	Data source
run_analysts	Fundamental, risk and ownership checks; technical analysis and a chart only when asked	Yahoo Finance
get_price	Live quotes and 1-day, 1-week, 1-month changes	Yahoo Finance

The RAG agent in simple terms

RAG means the model answers from retrieved documents instead of from memory. Two Pinecone indexes hold the same chunks of Apple and Rivian filings (10-K, 10-Qs, 8-Ks): one searches by meaning, one by exact words.

1. **Search twice:** the meaning search and the keyword search each return 30 candidates.
2. **Merge:** the two lists are combined (reciprocal rank fusion).
3. **Rerank:** a reranker model scores each passage against the question and keeps the top 5.
4. **Cut off:** passages scoring below 0.2 are dropped.
5. **Grade:** the grader agent keeps only passages that really answer the question.
6. **Retry once:** if nothing survives, the rewriter rewords the query in filing language and the search runs again.

Filing questions always run this search first; the analysts are used only if it finds nothing (outlook questions are the exception).

Models used

Two kinds of model are involved: **chat LLMs**, which read and write text, and **retrieval models**, which turn text into numbers or score relevance. Every chat call runs at temperature 0 (as repeatable as possible) and must return a fixed JSON format.

Used by	Model	Kind	Provider	Limits
Gate	`typesafe/jev-router`	Chat LLM (auto-router: picks an underlying model per call, for example GLM 5.3 Flash or DeepSeek V4.1 Flash)	OpenRouter	1,000 output tokens; one retry; no fallback
Planner, tool-calling agent, grader, rewriter	`gpt-4o-mini`	Chat LLM	OpenAI	1,000 output tokens
Writer	`gpt-4o-mini`	Chat LLM	OpenAI	1,500 output tokens; compact JSON; one retry
Semantic search	`llama-text-embed-v2`	Embedding model (1,024 numbers per chunk)	Pinecone	Long text cut at the end
Keyword search	`pinecone-sparse-english-v0`	Sparse word-matching model	Pinecone	512 tokens per chunk
Reranking	`bge-reranker-v2-m3`	Cross-encoder: scores question and passage together, 0 to 1	Pinecone	1,024 tokens per pair, cut at the end

The OpenAI model can be changed with `OPENAI_MODEL`; the gate model with `ROUTER_MODEL`. Without OpenRouter settings the gate runs on the OpenAI model.

Vector database and indexes

The project uses one vector database, **Pinecone** (serverless, AWS us-east-1), with two indexes that hold the same chunks. Pinecone embeds the text itself when chunks are uploaded and when a question is searched.

	filings-dense	filings-sparse
Searches by	Meaning	Exact words
Dimension	1,024	None (sparse vectors have no fixed size)
Similarity	Cosine	Dot product
Records	4,395	4,395 (same chunks)
Namespaces	`semantic` 2,881 · `fixed` 1,514	Same
Companies	AAPL (663 semantic, 418 fixed) · RIVN (2,218 semantic, 1,096 fixed)	Same

What is stored. For each company: the latest 10-K, the last three 10-Qs and 8-Ks from the last 90 days, downloaded from SEC EDGAR. Each record holds the chunk text plus `ticker`, `form`, `filing_date`, `fiscal_year`, `section`, `chunk_type` and `source_url`, with an id such as `AAPL-10-K-2025-10-31-Document-0`. Searches are filtered by ticker. Holdings are not stored here; they stay in memory.

Chunking and overlap

Filings are too long to send whole, so they are cut into chunks. Two rules apply to both methods: a **table is never cut** (it stays one chunk, so row labels stay with their numbers), and a **chunk never crosses a section** (so every chunk belongs to one item, such as Item 1A or Item 7). Both chunk sets are stored, each in its own namespace.

	Semantic chunking (used by the app)	Fixed-size chunking (kept for comparison)
How it cuts	Embeds each sentence and starts a new chunk when the topic changes: similarity to the chunk so far drops below 0.6	Every 500 words
Size	At least 100 words before a topic cut; never more than 1,000 words	500 words

	Semantic chunking (used by the app)	Fixed-size chunking (kept for comparison)
Overlap	None: cuts fall between topics, so nothing is split mid-thought	50 words repeated between neighbouring chunks, so a sentence near a cut is whole in at least one chunk
Namespace	`semantic` (2,881 chunks)	`fixed` (1,514 chunks)

The app searches the namespace named by `RAG_NAMESPACE` (default `semantic`). The keyword index reads only the first 512 tokens of a chunk, so it misses the tail of the longest semantic chunks.

Search settings

Setting	Value	Meaning
Candidates	30 per index	How many passages each search returns before merging
Rank fusion k	60	Merges the two lists: each passage scores 1 / (60 + rank) from each list
Top N	5	Passages kept after reranking
Relevance cutoff	0.2	Reranker score below which a passage is dropped
Grader input	1,500 characters	How much of each passage the grader reads
Query rewrites	1	Retries with filing vocabulary when nothing relevant was found

These are starting values and were not tuned on this project's data.

Guardrails

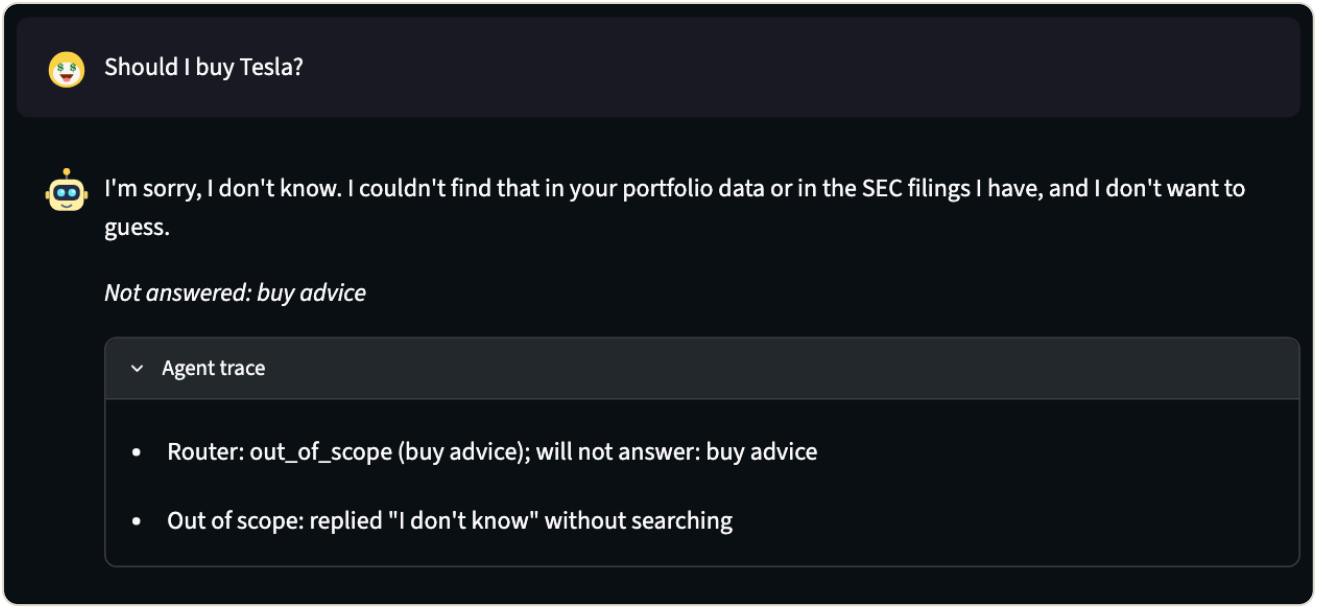
Guardrails are the rules, mostly enforced in code, that stop the assistant from guessing or misleading.

Citations are checked	A reply citing a source that was never supplied is rejected (one retry, then “I don't know”).
Numbers are checked	Figures that appear in no source are listed under “What I couldn't verify”.
No arithmetic by the LLM	All sums, weights and stress tests are computed in code.
Out-of-scope refusal	Advice, price predictions and news are declined; a mixed question answers only its in-scope part and says what it skipped.
Unknown companies refused in code	A company name with no real ticker (for example “Vivian”) is refused, so it can never borrow your portfolio's numbers.
Outlook only from data	The outlook section may cite only analyst facts, and ends with “I'm not a financial advisor”.
Untrusted documents	Instructions hidden inside filing text are ignored.
Bounded effort	Caps on tool calls, turns, rewrites and output length; a runaway reply is retried once.
Transparent trace	Every reply has an Agent trace showing the path, each tool call and its time.

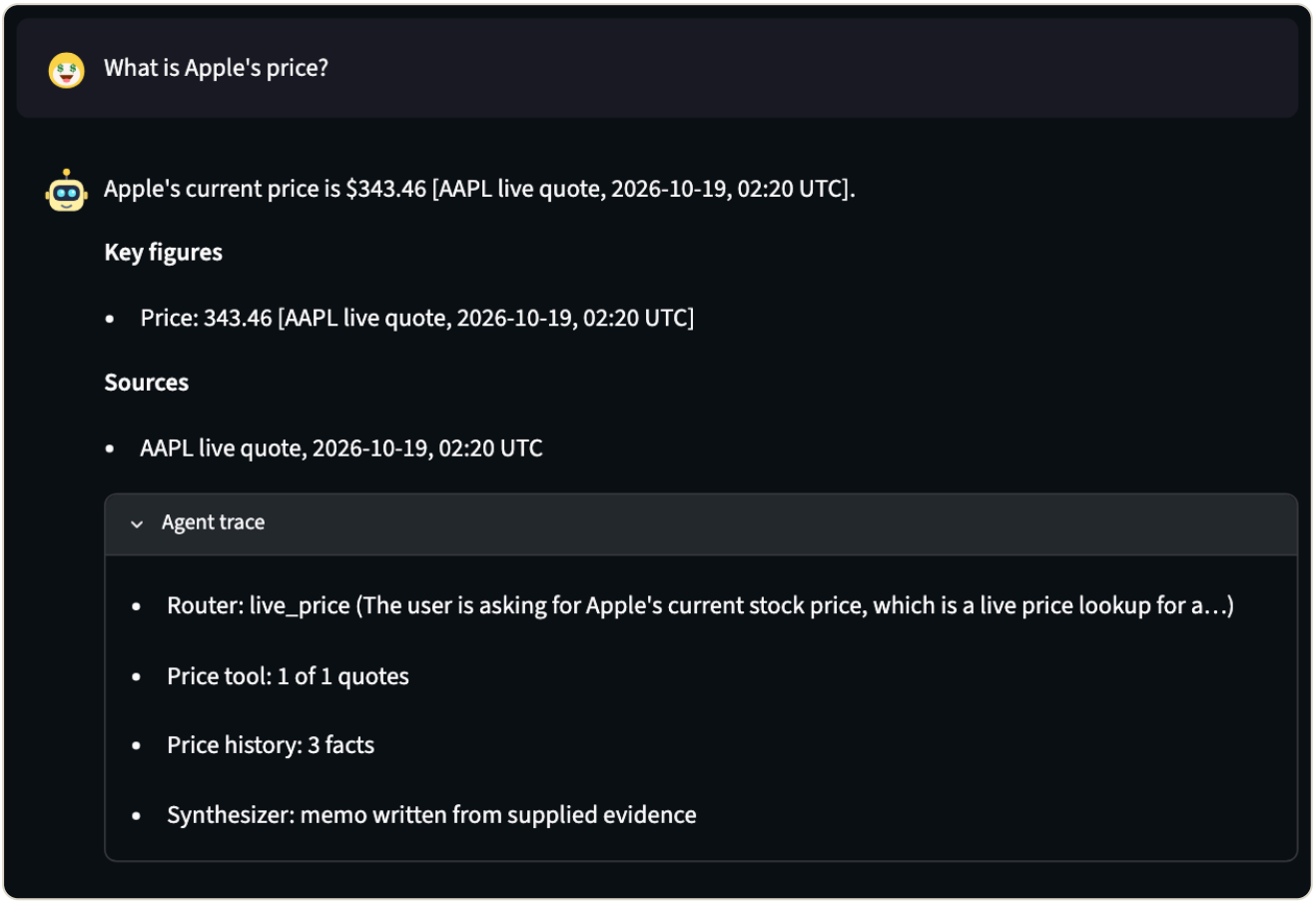
The four paths in the app

Screenshots of the running app (sample portfolio, 19 October 2026), each with its Agent trace open to show the path taken. Live prices change, so figures will differ.

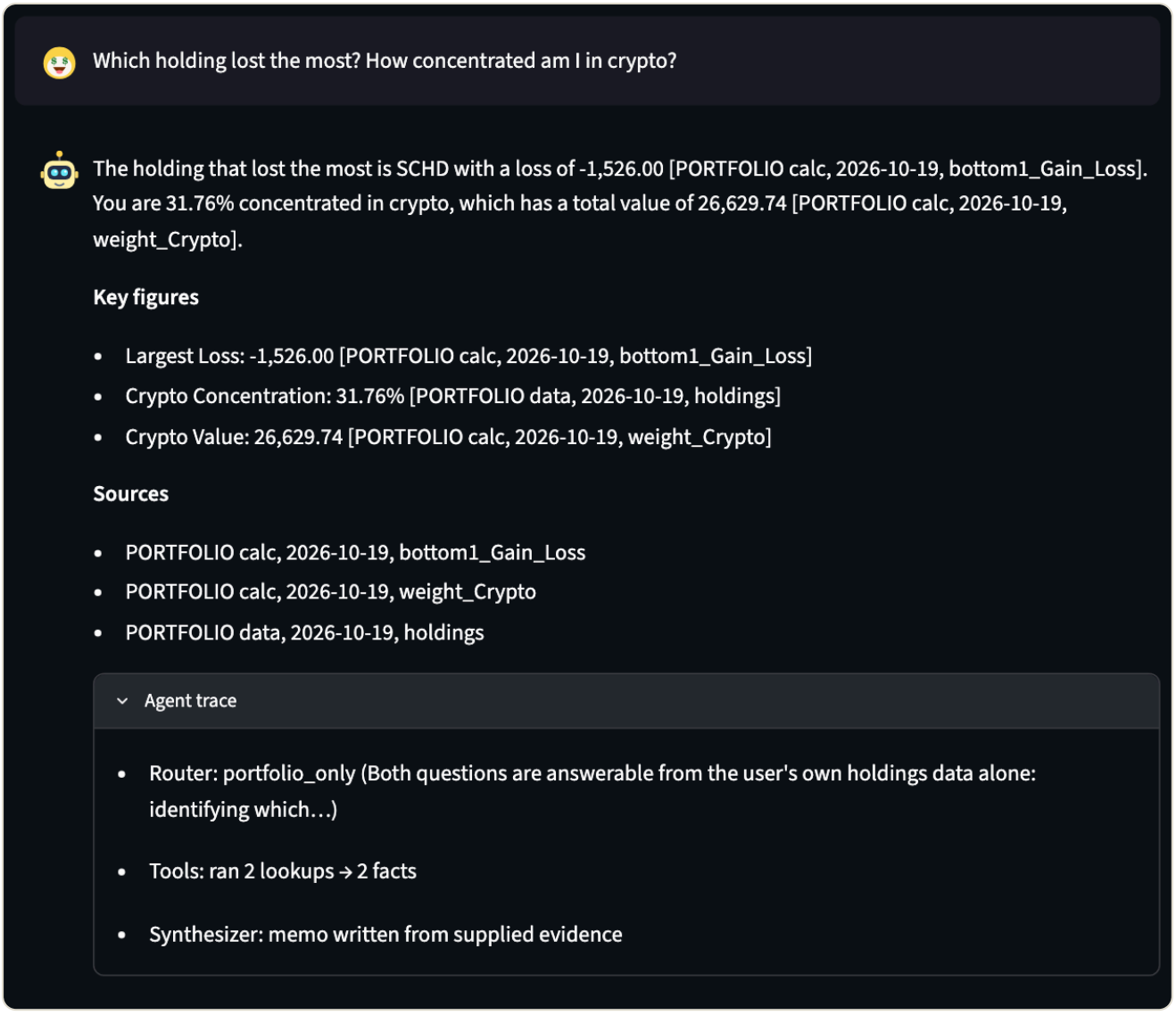
`1 · refuse` “Should I buy Tesla?” The gate routes it `out_of_scope`: no search runs, and the reply names the part it won't answer.



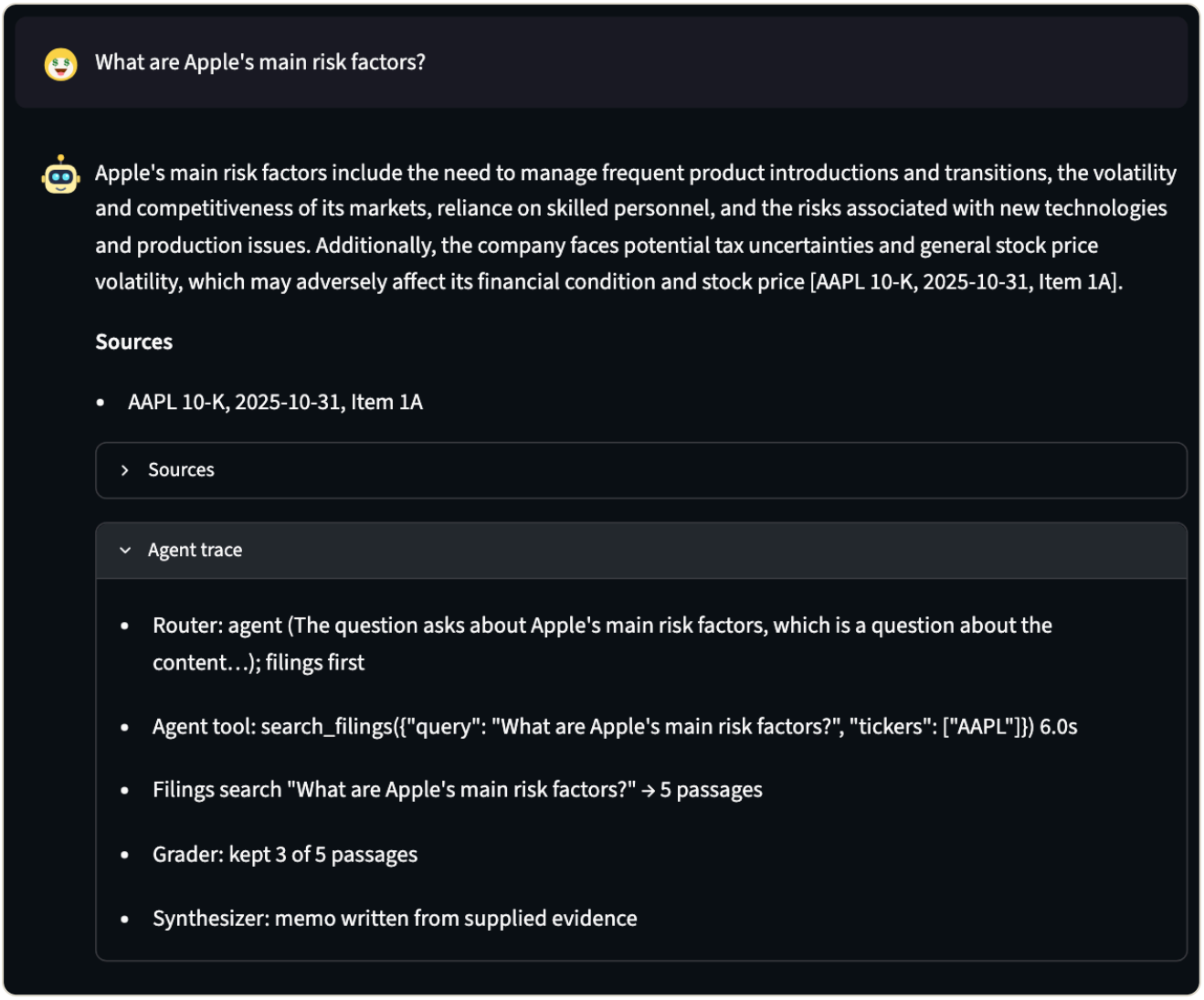
`2 · live price` “What is Apple's price?” The gate routes it `live_price`; the price tool returns a timestamped quote and the writer cites it.



`3 · portfolio` “Which holding lost the most? How concentrated am I in crypto?” The gate routes it `portfolio_only`; the planner runs 2 calculations (SCHD lost 1,526.00; crypto is 31.76%).



`4 · agent` “What are Apple's main risk factors?” The gate routes it `agent` with filings first; the RAG search finds 5 passages, the grader keeps 3, and the answer cites the 10-K's Item 1A.



Before you start: the portfolio file

Ask AI answers about the portfolio loaded in the app, so a holdings file is a prerequisite. Upload your own CSV in the sidebar, or switch on **Use Sample Dataset** to load the test file that ships with the project: `data/sample_portfolio.csv` (12 holdings across stocks, ETFs, crypto and REITs).

Column	Rule	Example
Ticker	Required; Yahoo Finance symbol, made upper case. Crypto uses Yahoo symbols.	`AAPL`, `BTC-USD`
Shares	Required; a number greater than 0	`15`, `0.25`

Column	Rule	Example
Purchase_Price	Required; a number, 0 or more	`150.25`
Purchase_Date	Required; a readable date	`2023-01-15`
Asset_Class	Required; any label. Used for weights and what-if shocks.	`Stocks`, `ETFs`, `Crypto`, `REITs`

```
Ticker,Shares,Purchase_Price,Purchase_Date,Asset_Class
AAPL,15,150.25,2023-01-15,Stocks
SCHD,40,72.50,2023-04-18,ETFs
BTC-USD,0.25,28500.00,2023-06-10,Crypto
O,35,62.10,2023-01-30,REITs
```

Rows that break a rule are dropped, and the app lists their line numbers. Extra columns are ignored. A file with none of the required columns, or no valid rows, is rejected. Live prices come from Yahoo Finance; a ticker without a quote is flagged and left out of totals.

How to review and evaluate it

```
uv sync                                          # install
uv run streamlit run app.py                      # run the app
uv run pytest                                    # 364 offline tests, no keys need
uv run python -m agent.eval.run_router_eval      # gate accuracy on 49 labelled qu
uv run python -m agent.eval.run_agent_eval       # end-to-end checks on 9 question
```

Latest results: the gate scored 100% on 49 labelled questions and the end-to-end check passed 9 of 9. Live checks call OpenAI, OpenRouter and Pinecone, so results can vary slightly between runs.

Setup details (keys, the filing index, troubleshooting) are in `docs/ASK_AI.md` in the repository.

Known limitations

Two companies indexed	Only Apple and Rivian filings can be searched; other tickers get a “no filings available” note.
No news source	Questions about recent news are declined by design.

Search wording	Casual wording can miss the right filing passage (for example Rivian's revenue drivers), giving “I don't know”.
Model variability	The gate is an auto-router, so a rare question can be routed differently between runs.
Unofficial market data	Yahoo Finance fields can be missing; they show as “unavailable”.
Speed	Agent-path answers take 10–17 seconds.

Ask AI is educational analysis, not financial advice.